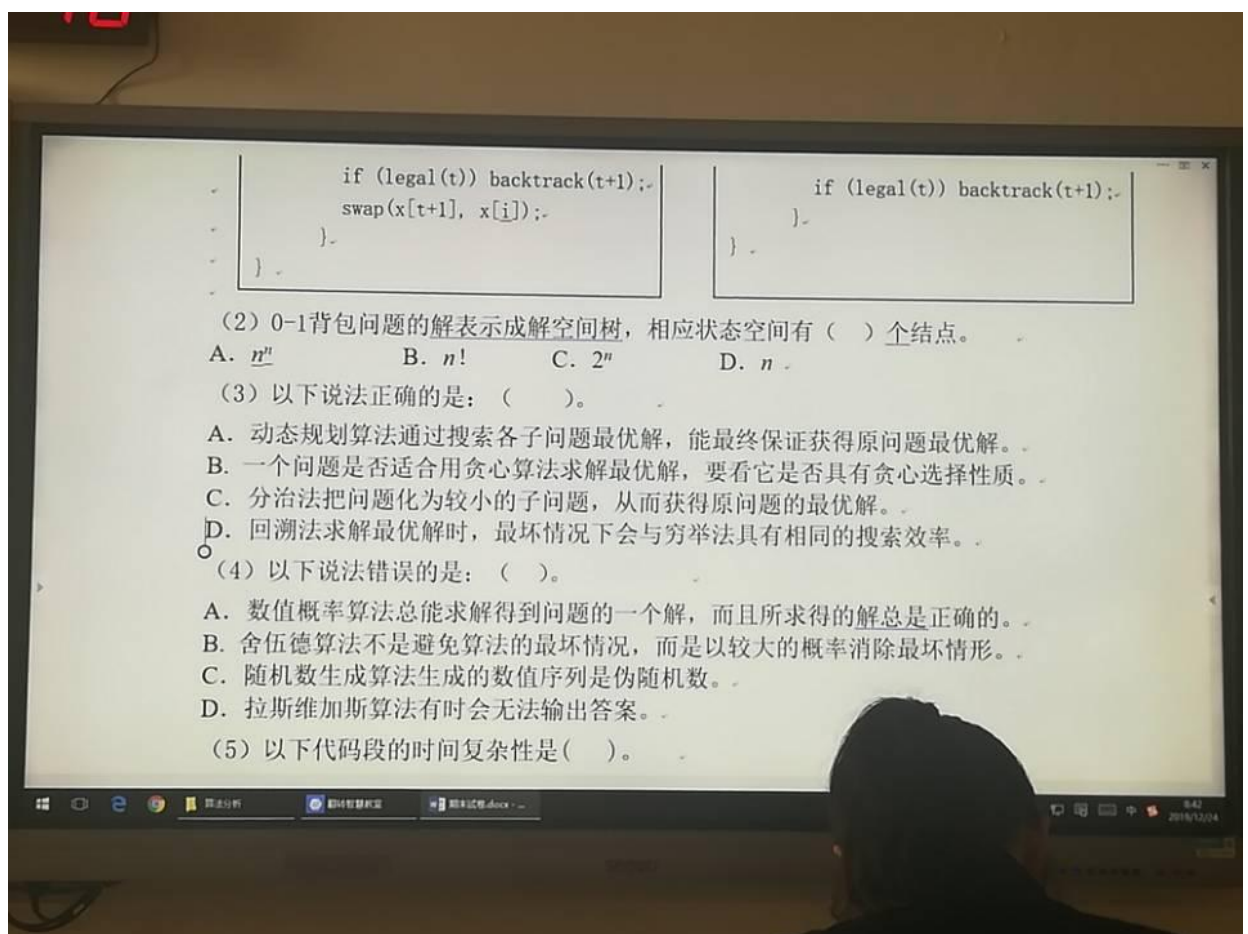
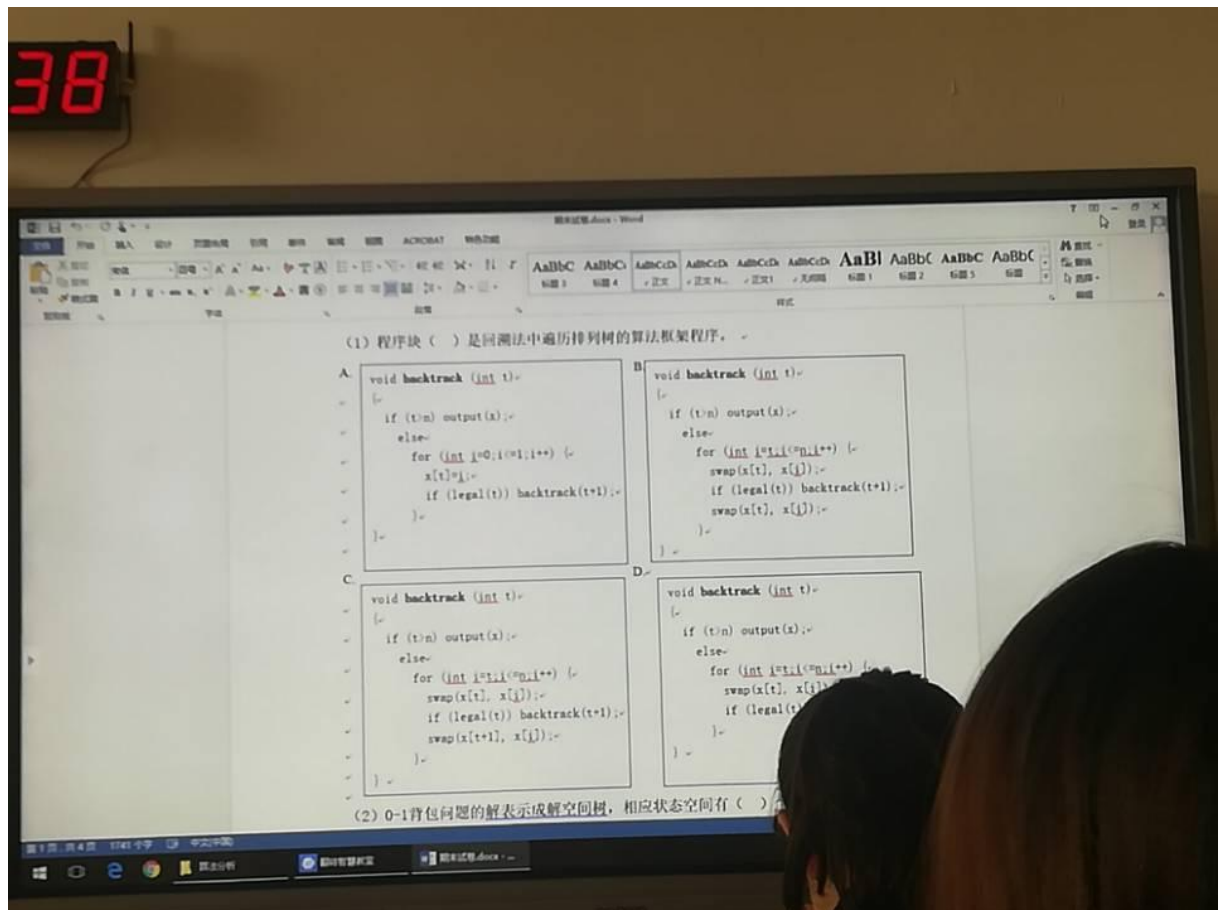




0857

- C. 分治法把问题化为较小的子问题，从而获得原问题的最优解。
D. 回溯法求解最优解时，最坏情况下会与穷举法具有相同的搜索效率。

(4) 以下说法错误的是：()。

- A. 数值概率算法总能求解得到问题的一个解，而且所求得解总是正确的。
B. 舍伍德算法不是避免算法的最坏情况，而是以较大的概率消除最坏情形。
C. 随机数生成算法生成的数值序列是伪随机数。
D. 拉斯维加斯算法有时会无法输出答案。

(5) 以下代码段的时间复杂度是()。

```
for (j=1; j<=n; j++)
    for (k=n; k>=1; k/=2)
        count++;
```

- A. $O(n^2)$ B. $O(n \log n)$ C. $O(\log n)$ D. $O(n)$

(6) 分支限界法与回溯法的不同点不包括()。

- A. 构造的解空间树不同； B. 搜索方式不同；
C. 对扩展结点的扩展方式不同； D. 存储空间的要求不同。

(7) 设 $T(n)=n$ ，根据 $T(n)=O(f(n))$ 的定义，下列等式不成立的是()。

- A. $T(n)=O(n^2)$ B. $O(n^2)=T(n)$

0859

(7) 设 $T(n)=n$ ，根据 $T(n)=O(f(n))$ 的定义，下列等式不成立的是()。

- A. $T(n)=O(n^2)$ B. $O(n^2)=T(n)$
C. $T(n)=O(\log n)+O(n)$ D. $T(n)=O(n) * O(\log n)$

(8) 下列算法中不能解决0/1背包问题的是()。

- A. 分治法 B. 动态规划 C. 回溯法 D. 分支限界法

(9) 当输入规模为 n 时，算法增长率最大的是()。

- A. 5^n B. $20 \log_2 n$ C. $2n^2$ D. $3n \log_2 n$

(10) $T(n)$ 表示当输入规模为 n 时的算法效率，以下算法效率最优的是()。

- A. $T(n)=T(n-1)+1, T(1)=1$ B. $T(n)=2n^2$
C. $T(n)=T(n/2)+1, T(1)=1$ D. $T(n)=3n \log_2 n$

(11) 以下对算法的描述，不正确的是()。

- A. 动态规划和贪心两种算法都是基于分治的思想。
B. 分治法要求分解的子问题和原问题相比，问题规模更小但性质相同。
C. 程序与算法的主要区别在于算法要求有穷性，程序不需要。
D. 动态规划算法采用自顶向下的方式来逐步解决一个问题。

(12) 下述表达不正确的是()。

1900

(15) 在使用回溯法解决八皇后问题时，避免无谓搜索的策略为使用 ()。

- A. 递归函数 B. 约束函数 C. 限界函数 D. 搜索函数

二、简答题 (3 小题，共计 15 分)。

1. 用 O 、 Ω 、 Θ 表示函数 f 与 g 之间的关系， b 表达为 $f = ()g$ 。(本小题 6 分)。

(1) $f(n) = \log_2 n$, $g(n) = 200\sqrt{n}$;

(2) $f(n) = n-1$, $g(n) = 2\log_2 n \times \sqrt{n}$;

(3) $f(n) = 3^n$; $g(n) = n!$;

2. 简述蒙特卡罗算法的适用范围和特点，以及算法设计要求。(本小题 4 分)。

3. 设在通信中只可能出现 8 种字符，其出现的概率分别为 0.05, 0.29, 0.07, 0.14, 0.23, 0.03, 0.11。试设计霍夫曼编码，并画出编码树。(本小题 4 分)。

三、代码填空题 (每空 2 分，共 14 分)。

1. 活动安排问题。

void GreedySelector(int n, Tvne s[], Tvne f[], bool A[])

```
int func(string str1, string str2)
{
    vector<int> tmp(str2.size()+1);
    vector<vector<int>> dp(str1.size()+1,tmp);
    for (auto i = 1; i <= str1.size(); ++i)
    {
        for (auto j = 1; j <= str2.size(); ++j)
        {
            dp[i][j] = 1;
            dp[i][j] = 2;
            if (3)
            {
                dp[i][j] = max(dp[i][j], dp[i-1][j-1] + 1);
            }
            else
            {
                dp[i][j] = max(dp[i][j], dp[i-1][j-1]);
            }
        }
    }
    return 4;
}
```

```

    }
    return 4;
}

```

四、算法设计题（共 32 分）

1、在黑板上写了 N 个正整数作成的一个数列，进行如下操作：每一次擦去其中的两个数 a 和 b ，然后在数列中加入一个数 $a \times b + 1$ ，如此下去直至黑板上剩下一个数。在所有按这种操作方式最后得到的数中，最大的记作 $\max$ ，最小的记作 $\min$ ，该数列的极差定义为 $M = \max - \min$ 。请写出算法计算极差。（本小题 10 分）

2. 有 8 个份额的资源，分配给 3 个工程。利润函数 $G_i(x)$, $1 \leq i \leq 3, 0 \leq x \leq 8$ 表示 x 份资源分配给第 i 个工程所得到的利润，其利润函数

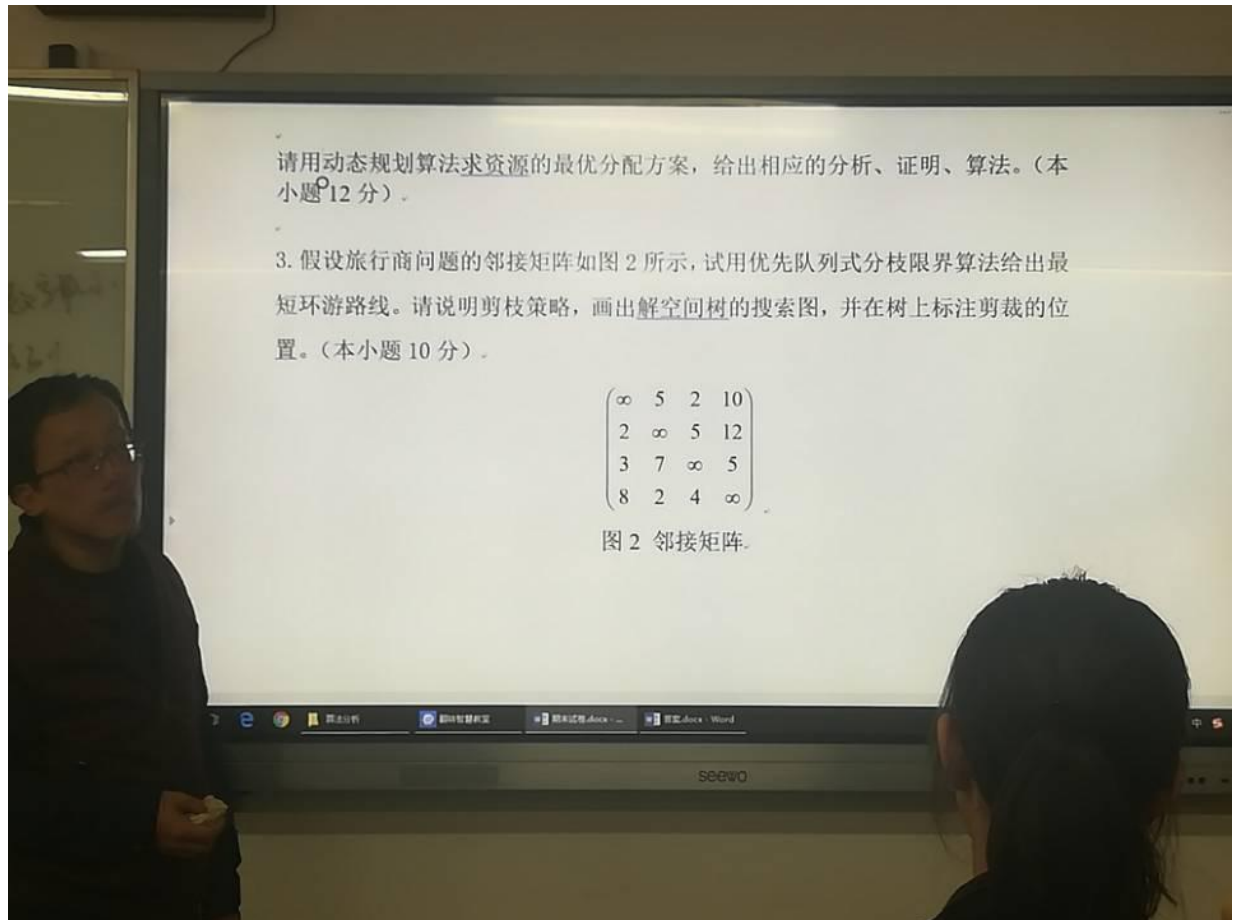
数。在所有按这种操作方式最后得到的数中，最大的记作 $\max$ ，最小的记作 $\min$ ，该数列的极差定义为 $M = \max - \min$ 。请写出算法计算极差。（本小题 10 分）

2. 有 8 个份额的资源，分配给 3 个工程。利润函数 $G_i(x)$, $1 \leq i \leq 3, 0 \leq x \leq 8$ 表示 x 份资源分配给第 i 个工程所得到的利润，其利润函数如图 1 所示。

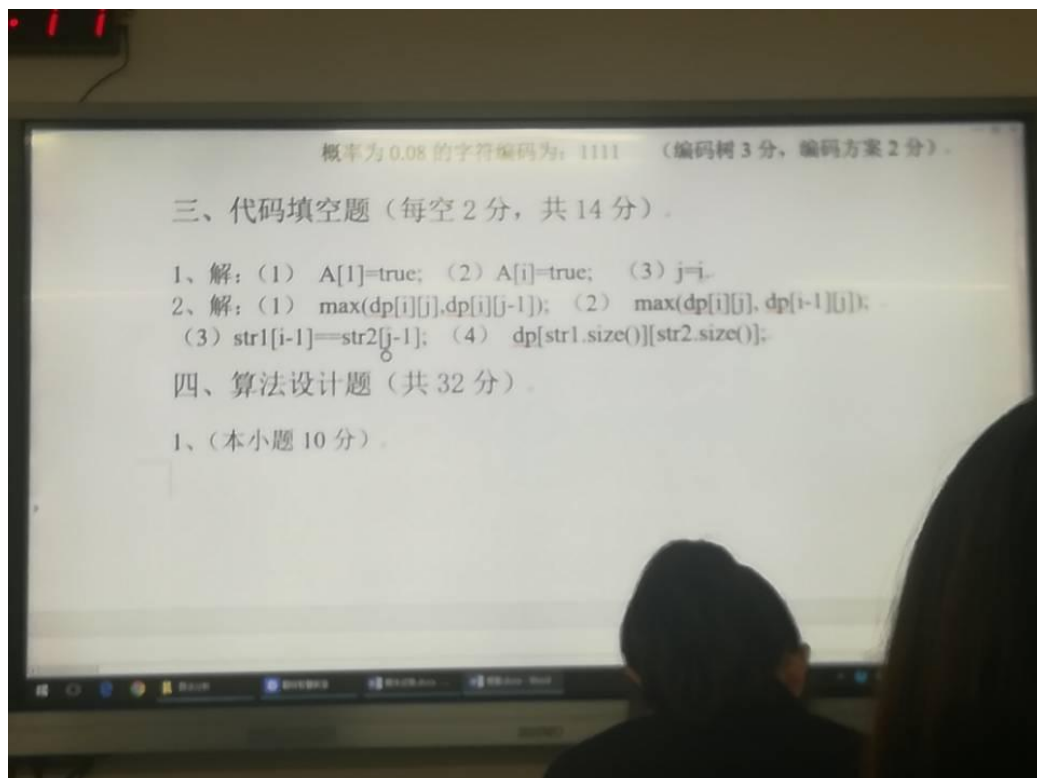
x	0	1	2	3	4	5	6	7	8
$G_1(x)$	0	4	26	40	45	50	51	52	53
$G_2(x)$	0	5	15	40	60	70	73	74	75
$G_3(x)$	0	5	15	40	80	90	95	98	100

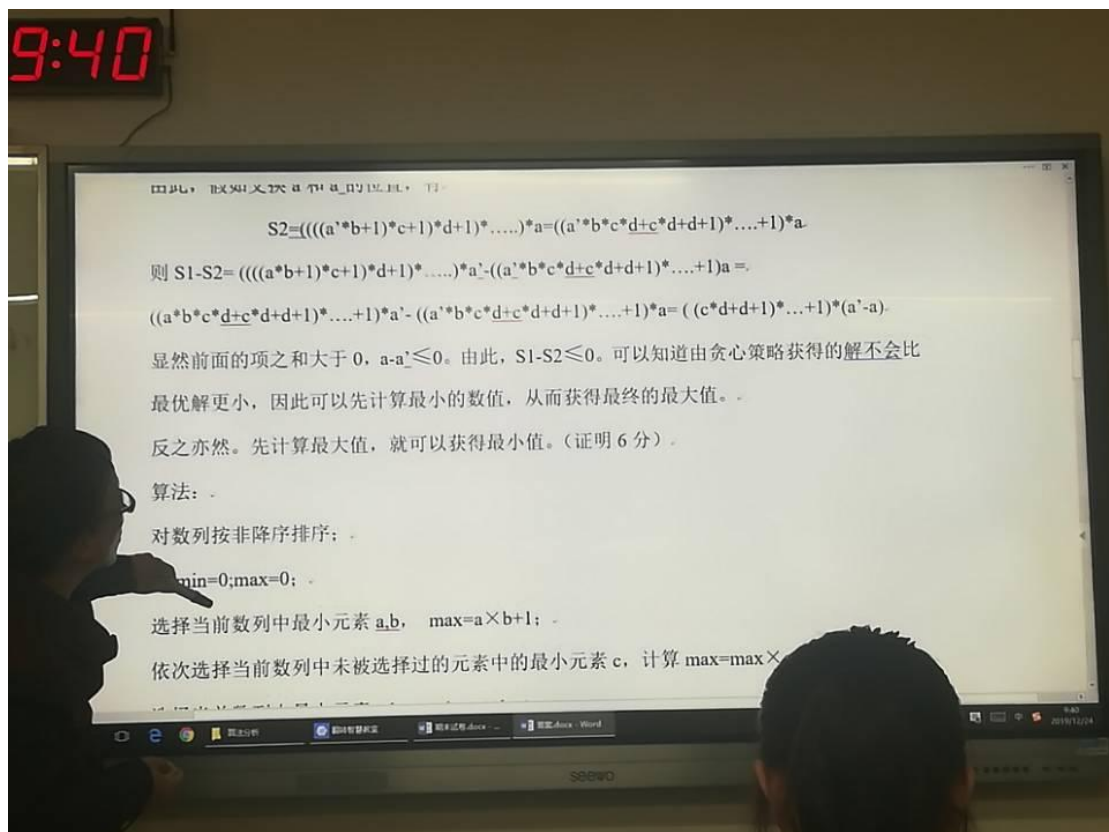
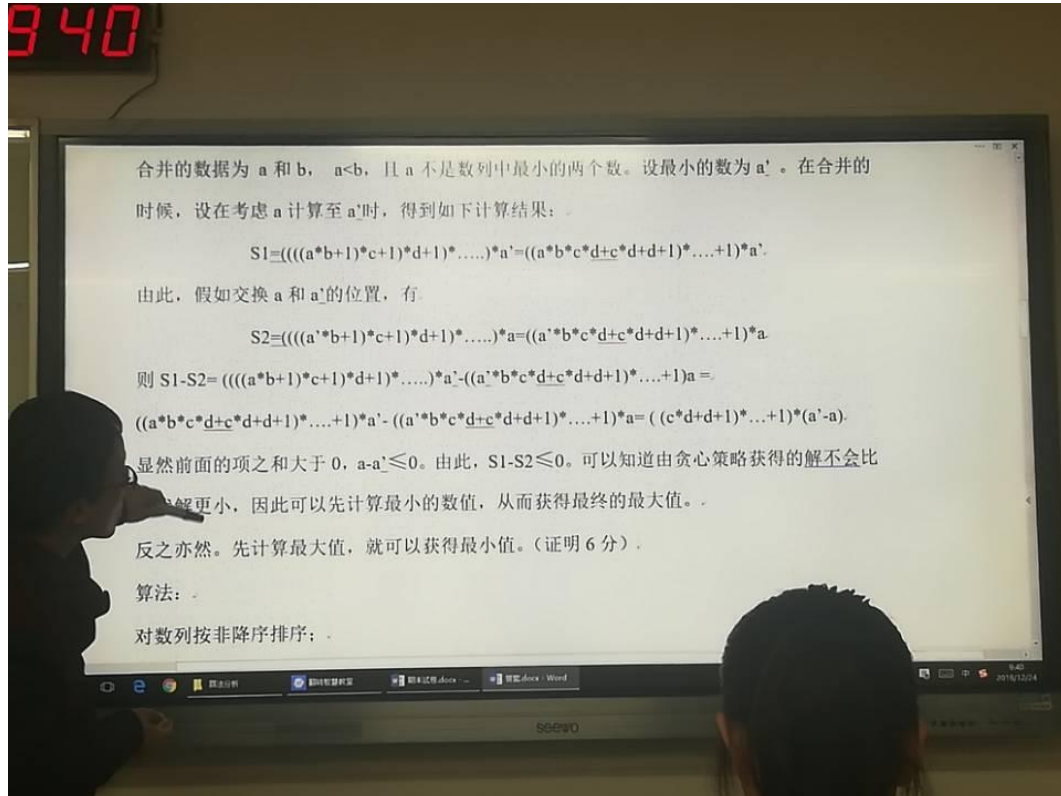
图 1 三个工程利润图

请用动态规划算法求资源的最优分配方案，给出相应的分析、证明、算法。（本小题 12 分）



答案：





940

最优解更小，因此可以先计算最小的数值，从而获得最终的最大值。

反之亦然。先计算最大值，就可以获得最小值。（证明 6 分）。

算法：

对数列按非降序排序；

令 $\min=0; \max=0$ ；

选择当前数列中最小元素 a, b ， $\max=a \times b + 1$ ；

依次选择当前数列中未被选择过的元素中的最小元素 c ，计算 $\max=\max \times c + 1$ ；

选择当前数列中最大元素 a, b ， $\min=a \times b + 1$ ；

依次选择当前数列中未被选择过的元素中的最大元素 c ，计算 $\min=\min \times c + 1$ ；

计算极差 $M=\max-\min$ （算法描述 4 分）。

2. （本小题 12 分）。

解： $G(m) = \sum_{j=1}^n G_j(x_j)$ ， $\sum_{j=1}^n x_j = m$ ，其中 n 表示资源数， m 表示工程

2. (个分) (12分)

解: $G(m) = \sum_{i=1}^n G_i(x_i)$ $\sum_{i=1}^n x_i = m$, 其中 n 表示资源数, m 表示工程数。

最优子结构性质的证明: 由反证法可得。假设存在一个最优解 P_0 。在该方案中, 第 1 个工程获得 k 份资源, 子问题为对 $n-k$ 份资源在 $m-1$ 个工程的分配方案。由于 $n-k$ 份资源的最优分配方案与第一个工程无关, 因此设原问题中, 对应子问题分配方案最优解为 P_1 , 如果在子问题中存在更优的分配方案 P_2 , 将使得 $P_0 = V[1][k] + P_1 \leq V[1][k] + P_2$, 从而与解的最优性矛盾。(证明 4 分)。

输出: 最优分配时所得到的总利润 optg, 最优分配时各项工程所得到的份额 optq[]。

```
template <class Type>
Type allot_res(int n,int m,Type G[],int optg[])
3. {
4.     int optx,k,i,j,s;
5.     int *q = new int[n];          /* 分配工作单元 */
6.     int (*d)[m+1] = new int[n][m+1];
7.     Type (*f)[m+1] = new Type[n][m+1];
8.     Type *g = new Type[n];
9.     for (j=0;j<=m;j++) {          /* 第一个工程的份额利润表 */
10.         f[0][j] = G[0][j];    d[0][j] = j;
11.     }
12.     for (i=1;i<n;i++) {           /* 前 i 个工程的份额 */
13.         f[i][0] = G[i][0] + f[i-1][0];
14.         d[i][0] = 0;
15.         for (j=1;j<=m;j++) {
```

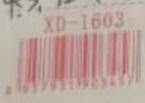

NAME --> ID
+ + + + +
Preliminary
C
+ + + + +
CBA
71
1. A
M
10. D, A

$v = v(g, w)$
定根无子树列正序
 $dp[i][j] = 0$
 $dp[i][j] = \text{set}$
 $\text{str}(i, j) = \text{set}$
~~for i in range(1, n+1)~~
 $dp[i].size$
 $\text{solve}(int\ a, n=1)$
找到所有子树
swa
swa
0-1 solve at
solve

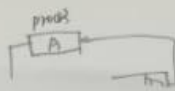
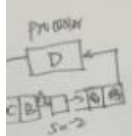
1. 回溯法中遍历排列树 $\rightarrow$ 交换 $\text{swap}[x[i], x[j]]$ break (return) swap (return) $\text{swap}[x[i], x[j]]$
遍历子集树 无需交换
2. 0-1背包问题 $\rightarrow$ 解空间树 2^n 个结点
3. A: 递归求得最优解 X 前提: 最优子结构性质
C: 分治法化为子问题 $\rightarrow$ 原问题最优解 X 无最优解 (子问题)
D: 回溯 最坏与穷举有相同的时间复杂度 $\checkmark$ 无剪枝
4. A $\rightarrow$ 近似解
5. $n \log n$
6. A 分支回溯 解空间树相同
7. B
8. A 为法不可解
9. A
10. D
11. D 为规则自底向上
13. B 限界 (找最优) 以剪枝应用

二. 1. 0, Ω , 0 $f = O(g)$ $f(n) = \log n$ $g(n) = 200 \sqrt{n}$
 Ω n^4 $2 \log_2 n \times \sqrt{n}$
 0 3^n $n!$

2. 蒙特卡罗适用范围特点
3. 霍夫曼编码



红线稿纸



EA 0
xj: 1~6

max 有文
min

慢有最优

角解. 算

2. 证

3. 证

4. 证

5. 证

6. 证

7. 证

8. 证

9. 证

10. 证

11. 证

12. 证

13. 证

14. 证

15. 证

16. 证

17. 证

18. 证

19. 证

20. 证

三. 源代码填充

1. 代码安排 流动安排

2. 最长公共子序列

$dp[i][j] = \max$

$str[i] == str[j]$

$dp[str1.size()][str2.size()]$

四. 计算板差

擦去其中的2个数 a, b 加 $a \times b + 1$

写证明.

$a \times b + 1$

a, b 最优子结构

1 2 3 3 4 5

2 / 10 21 211 11 min 证明

20

1 8 4 6 1

x 2 368 369 370 11 max 证明

1 2 3 4 5 6 125

34

376

1 2 3 4 5 6

65

11 max 证明 最优子结构

两边

x y

$x + y + 1$

最小

$(x \times y + 1)$

子问题减2个数+1个致形成子问题最小